

Organización de Datos

Trabajo Práctico N°4

Por:

Mario Arriaga, Gabriel Seneca, Mia Casanovas Di Sera y Augusto Sulsente

6)

El ejercicio se dividió en exactamente dos partes: Clásico y por Montículos. Para el caso del ejercicio clásico, se definió un nodo de árbol binario, el cual contiene una clave (en este caso, un número entero) y dos punteros, uno al hijo izquierdo y otro al hijo derecho; en caso de no existir alguno, el campo se establece en NULL.

```
typedef struct nodo{
    int dato;
    struct nodo *izq, *der;}nodo;
typedef nodo * TArbolBinario
```

Luego, para la inserción se aprovechó la naturaleza de los árboles binarios: se comienza por la raíz y se compara la clave con el nodo actual; si es menor, se continúa recorriendo por la rama izquierda, de lo contrario, por la derecha. Se pueden destacar dos casos particulares:

- Si alguno de los dos punteros es NULL, significa que la nueva clave debe insertarse ahí, por lo que se crea un nodo nuevo con los punteros a hijos en NULL y con el dato correspondiente a la clave a insertar.
- Cuando la clave es igual al dato del nodo, se decidió, como convención, seguir por la rama derecha.

```
void inserción(TArbolBinario AB, int clave){(1)
    TArbolBinario nuevo;
    if(AB!=null){(2)
        if(AB->dato < clave)
            if(AB->izq != null)
                inserción(AB->izq,clave)
            else{(3)
                nuevo = (TArbolBinario)malloc(sizeof(nodo));
                nuevo->dato = clave;
                nuevo->izq=nuevo->der=null;
                AB->izq=nuevo;
            }(3)
        else{(4)
            if(AB->der != null)
                inserción(AB->der,clave)
            else{(5)
                nuevo = (TArbolBinario)malloc(sizeof(nodo));
                nuevo->dato = clave;
                nuevo->izq=nuevo->der=null;
                AB->der=nuevo;
            }(5)
        }(4)
    }(2)
}(1)
```

Este mismo procedimiento se aplicó también para la búsqueda. Habiendo definido estas funciones, se intentó insertar un total de 100.000 claves, de lo cual se extrajeron dos conclusiones principales:

1. Debido a la naturaleza recursiva de las funciones, aunque facilita su desarrollo, también incrementa progresivamente el tiempo de ejecución.
2. Por el mismo motivo, al llegar a un número extremadamente grande de claves, pero antes de alcanzar el objetivo, el programa crasheó por stack overflow.

Utilizando montículos:

Utilizamos el montículo binario, donde el nodo raíz es el mínimo.

El montículo se define como un arreglo ya que de esta manera se implementa con más eficiencia (compacidad, localidad de memoria, operaciones indexadas sencillas).

Estructura del heap:

```
typedef struct {  
    int *arr; // arreglo dinámico que contiene los valores  
    int size; // cantidad actual de elementos  
    int capacidad; // capacidad reservada  
} Monticulo;
```

Funciones implementadas y sus responsabilidades:

insertar(Monticulo *m, int valor) – insertar al final. Para insertar un nuevo elemento se sitúa al final del vector (última hoja del árbol) y se asciende hasta que se cumpla la propiedad.

```
void insertar(Monticulo *m, int valor) {  
    if (m->size == m->capacidad) {  
        int nueva = m->capacidad * 2;  
        int *tmp = (int*) realloc(m->arr, sizeof(int) * nueva);  
        if (!tmp) {  
            perror("realloc"); //si no hay más capacidad  
            // aquí podríamos manejar el fallo de forma más elegante  
            exit(EXIT_FAILURE);  
        }  
        m->arr = tmp;  
        m->capacidad = nueva;  
    }  
    m->arr[m->size] = valor;  
    ascender(m, m->size);  
    m->size++;  
}
```

ascender(Monticulo *m, int i) – comparar con el padre y subir si corresponde (mantiene propiedad del heap).

```

void ascender(Monticulo *m, int i) {
    while (i > 0) {
        int padre = (i - 1) / 2;
        if (m->arr[i] < m->arr[padre]) {
            swap(&m->arr[i], &m->arr[padre]); \\swap intercambia el padre con el
            nodo a ascender, es decir, "sube" el nodo si es que corresponde
            i = padre;
        } else break;
    }
}

```

buscarMonticulo *m, int valor) – búsqueda lineal $O(n)$ recorriendo el arreglo.

```

int buscar(Monticulo *m, int valor) {
    for (int i = 0; i < m->size; ++i) if (m->arr[i] == valor) return 1;
    return 0;
}

```

inorden(Monticulo *m, int i) – recorrido recursivo simulando árbol en arreglo (izq: $2*i+1$, raíz: i, der: $2*i+2$).

```

void inorden_limitado(Monticulo *m, int i, int *contador, int limite) {
    if (i >= m->size || *contador >= limite) return;
    inorden_limitado(m, 2*i+1, contador, limite);
    if (*contador < limite) {
        printf("%d ", m->arr[i]);
        (*contador)++;
    }
    inorden_limitado(m, 2*i+2, contador, limite);
}

```

Decimos inorden limitado ya que no devuelve una lista ordenada en montículos. (En la conclusión se profundiza)

Para la inserción de 100.000 claves aleatorias se reservó la capacidad necesaria desde el inicio para evitar realloc durante la carga (esto reduce costes y fragmentación). Con esta elección no ocurre realloc durante la inserción de exactamente 100.000 elementos. Luego, la inserción se implementó mediante una función de generación aleatoria de C. En cada inserción, se debe verificar que no haya overflow y crashee el programa. Esto es así ya que a pesar de reservar 100000 espacios de memoria, podría haber basura, y algunas celdas estar ocupadas previamente. Así, se genera la inserción de 100.000 elementos que a medida que se insertan en el último elemento ascienden a su posición de tener que hacerlo.

Conclusiones:

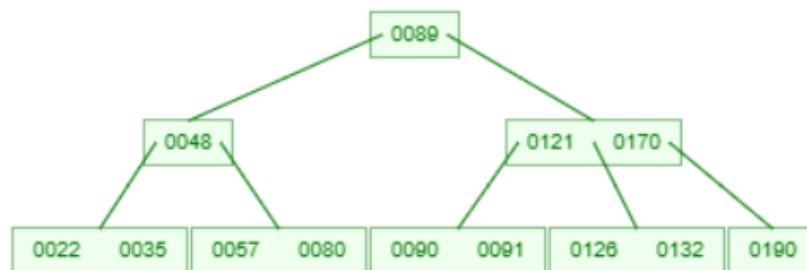
La implementación con arreglo es la más adecuada y eficiente para representar un montículo binario completo: ocupa memoria contigua, es cache-friendly y las operaciones típicas (insertar, extraer) son $O(\log n)$.

Para la consigna específica (inserción de 100.000 elementos), reservar la capacidad de antemano evita reubicaciones y es una elección práctica y eficiente.

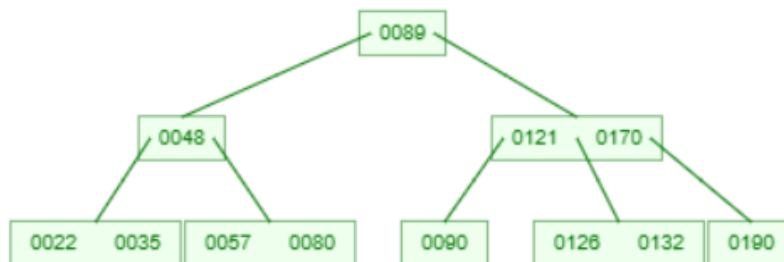
Es necesario aclarar que el recorrido inorden se implementó porque lo pedía la consigna, pero es fundamental entender que no devuelve una lista ordenada en montículos.

Si la consigna pidiera una búsqueda rápida y ordenación con orden como salida ordenada, entonces un ABB (preferiblemente balanceado) es la opción correcta. Si se requiere prioridad (extraer mínimo/ máximo) entonces Montículos es la opción correcta.

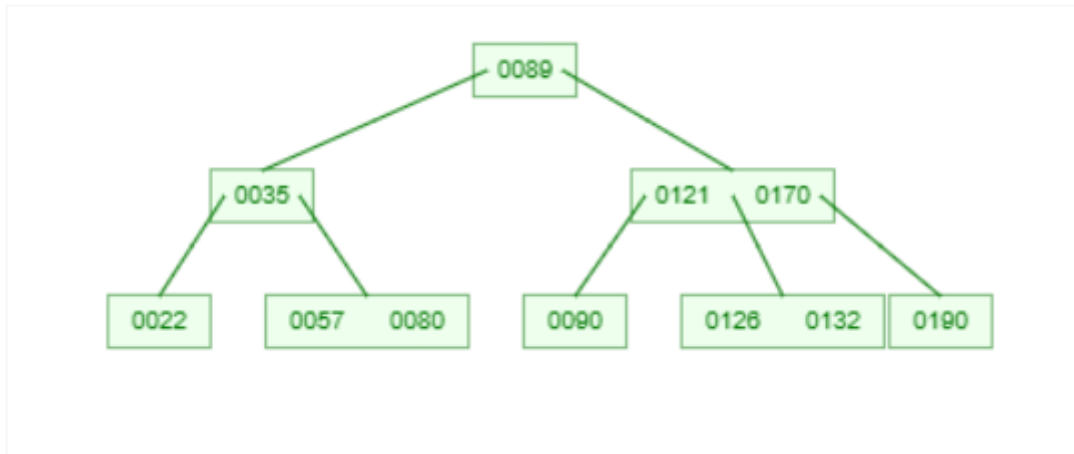
7) Dada la secuencia de claves enteras: 190, 57, 89, 90, 121, 170, 35, 48, 91, 22, 126, 132 y 80; dibuje el árbol B de orden 3 cuya raíz es R, que se corresponde con dichas claves.



8) En el árbol R del problema anterior, elimine la clave 91 y dibuje el árbol resultante.

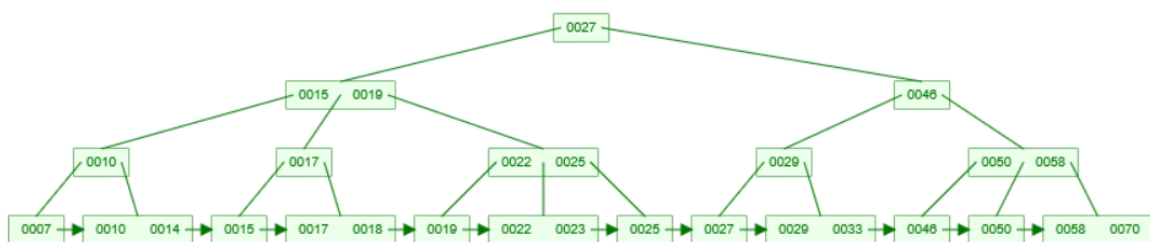


Elimine ahora la clave 48. Dibuje el árbol resultante, ¿se redujo el número de nodos?



No, el número de nodos no se redujo. Lo que sí se redujo fue el número de claves que contenían los nodos previamente al haber sido eliminadas las claves 48 y 91.

9)



10) Construir cada uno de los árboles B (B, B+ y B*) que se van generando conforme se van insertando los números 1, 9, 32, 3, 53, 43, 44, 57, 67, 7, 45, 34, 12, 23, 56, 73, 65, 49, 85, 89, 64, 54, 75, 77 en un árbol B de orden 5.

B:



B+:



B*:

